CS 4380 Project 1: 4380 Emulator Mark 1

NOTE: READ THIS DOCUMENT IN ITS ENTIRETY BEFORE YOU BEGIN WORKING

Introduction:

In this project you will begin the incremental development of a microprocessor emulator that you will build upon throughout the semester and utilize in CS 4490. This emulator simulates the execution of a hypothetical processor we are calling the 4380. At this stage you will implement the Mark1 version of the 4380, which has the following high-level architectural characteristics….

Instruction Size: Fixed at 12 Bytes (96 bits)

4 Byte Instruction	4 Byte Operand 1	4 Byte Operand 2

Registers (22):

Name	Encoding	Description (all registers are 32 bits in size)
R0 – R15	0 - 15	General purpose registers, **initialized to 0 at startup**
PC	16	Program Counter, **initialized to the value of first integer in memory**
SL	17	Stack Lower limit (lowest legal address available to the stack)
SB	18	Stack Bottom (highest address)
SP	19	Stack Pointer (next available location, grows downward)
FP	20	Frame Pointer (points to the bottom of the frame)
HP	21	Heap Pointer (initially set to SL, grows upward))

Memory: 32 bit address space; Byte addressable; Little-endian; 2^17 (131,072) Bytes of addressable memory

Instructions:

Jump Instructions:

Value	Operator	Operand 1	Operand 2	Description
1	JMP	Address	0	Jump to Address

Move Instructions:

Value	Operator	Operand 1	Operand 2	Description
7	MOV	RD	RS	Move contents of RS to RD
9	STR	RS	Address	Store integer in RS at Address
10	LDR	RD	Address	Load integer at Address to RD
11	STB	RS	Address	Store least significant byte in RS at Address
12	LDB	RD	Address	Load byte at Address to RD

Arithmetic Operations:

Value	Operator	Operand 1	Operand 2	Description
13	ADD	RD	RS	Add RS to RD, store result in RD
15	SUB	RD	RS	Subtract RS from RD, result in RD
16	MUL	RD	RS	Multiply RS by RD, result in RD
17	DIV	RD	RS	Divide RD by RS, result in RD

Traps/Interrupts:

Value	Operator	Operand 1	Operand 2	Description
21	TRP	0	0	Executes the STOP/Exit routine
21	TRP	1	0	Write int in R3 to stdout (console)
21	TRP	2	0	Read an integer into R3 from stdin
21	TRP	3	0	Write char in R3 to stdout
21	TRP	4	0	Read a char into R3 from stdin
21	TRP	98	0	Print all register contents to stdout

Requirements:

As you design and build your emulator you must adhere to the following requirements:

Req1) The emulator must reside in a PRIVATE GitHub repository called cs4380-emulator-assembler.

Req2) Project 1 work must be completed in a separate branch called project-1. When all requirements are met, the branch should be merged into main (BUT NOT DELETED).

Req3) The emulator must be written in C++.

Req4) The emulator shall NOT be implemented as a C++ class. The principle data structures and functions listed in these requirements shall have global visibility.

Req5) The project must utilize cmake/make so that the emulator can be built by invoking these two tools. The resulting executable shall be called **emu4380**.

Req6) The emulator must accept a single command line argument which shall be the name of a binary file containing 4380 byte code (e.g. binary encoded instructions that adhere to the requirements of the 4380 ISA as provided up to this point).

Req7) Upon being invoked with an appropriate input file the emulator shall read the contents of the binary file into its Program Memory. The first unsigned int's worth of Program Memory shall then be read into the PC register and execution shall commence. (See Req10)

Req8) The emulator shall implement a FETCH, DECODE, EXECUTE loop as its principal execution pattern. (See Req10)

Req9) Once execution begins the emulator shall continue to execute valid instructions until a TRP #0 instruction is encountered, at which point the emulator shall exit with a return value of 0. If at any point an invalid or unrecognized instruction is encountered the emulator shall print the message "INVALID INSTRUCTION AT: <address>" where <address> is the address in Program Memory of the first byte of the offending instruction. The emulator shall then exit with a return value of 1.

Req10) The emulator must consist of three C++ files: emu4380.h, emu4380.cpp, and main.cpp.

- emu4380.h shall contain function prototypes as well as declarations (not definitions) for the structural elements of the processor, including at a minimum:
    - A register file represented as an array of unsigned integers called **reg_file**. The general purpose and state registers of the processor shall be represented in order according to the encodings shown in the register table above (i.e. R0 shall be at position 0, SB shall be at position 18 etc.)
    - A program memory implemented as a 131,072 element array of unsigned char which shall be called **prog_mem.**
    - 3 control registers implemented as an array of 3 unsigned integers called **cntrl_regs** for storing decoded instruction information. These registers shall be utilized to store the instruction operation and first and second operands respectively.
    - Enum types for accessing/indexing the register file and control registers. The enum types shall be called **RegNames** and **CntrlRegNames**. The RegNames enum must follow the naming conventions utilized in the register table above, and should associate the appropriate encoding values to the register names.(i.e. R0:0, R1:1… R15:15, PC:16…HP:21). The CntrlRegNames enum must include the following naming associations: OPERATION:0, OPERAND_1:1, OPERAND_2:2. It is highly recommended (though not required) that enums be used to associate instruction names with their encoding values within the emulator as well.
- emu4380.cpp shall contain the processor element definitions and function definitions that simulate the execution phases of the 4380 processor, including FETCH, DECODE, and EXECUTE phases. These functions shall have the following prototypes (contained in the emu4380.h file):
    - void fetch(); // Retrieves the bytes for the next instruction and places them in the appropriate cntrl_regs. Also increments the PC to point to the next instruction.
    - bool decode(); // This function shall verify that the specified operation (or TRP) and operands as specified in the cntrl_regs are valid (i.e. a “known” instruction with legal operands). For example: a MOV instruction operates on state registers, and there are a limited number of these; a MOV instruction with an RD value of 55 would clearly be a malformed instruction. **NOTE:** Valid instructions are instructions that have been introduced thus far. The Mark 1 emulator shall ONLY accept the instructions described in this document. If an invalid instruction is encountered this function shall return false. Otherwise, it shall return true.
    - void execute(); // Executes the effects of the decoded and validated instruction on the state members (regs, memory, etc.) in accordance with

the instruction's specification. Alternatively executes the service routine for the specified TRP.

- main.cpp shall #include emu4380.h, and implement a main() function that processes a single command line argument. The single command line argument is assumed to contain a binary input file containing 4380 byte-code. The function shall read the contents of the file into program memory, initialize the PC register, and then enter an execution loop that calls the fetch(), decode(), and execute() functions() until a TRP 0 or illegal instruction are encountered.

Req11) The output of the TRP 1 and TRP 3 traps shall be printed without any leading or trailing whitespace of any kind.

Req12) The output of the TRP 98 trap shall be formatted as follows:

- One register name and value per line
- The register name shall be in all caps, followed by a tab character, followed by the register value printed as an unsigned base 10 integer. For example:

```
R1	0
R2	128
R3	34
…	…
HP	10045
```

Validation:

As always you are responsible for the validation of your project before submission. Before you begin working take the time to design your emulator. Identify the parts and pieces that will be needed and how they will need to interact. Then figure out how you can validate the behavior of these pieces both in isolation, and when integrated with the other parts of your system. Good modularity lends itself to good testing – and both lend themselves to well built, and maintainable code. Begin building up your test suite NOW, and automate as much of it as possible.

As a kindness and to assist you in your work we are providing a sample binary file called **HelloWorld.bin** which contains byte-code for a program that generates the following output:

Hello world
6 + 6 = 12
2 - 6 = -4
2 * 3 = 6
5 / 2 = 2
Your VM is working
Goodbye

We remind you that this is only a sample; it is only meant to HELP you validate your code and should not be considered exhaustive in any way.

What to Turn In:

When you are completed with the project you will need to submit the URL for your repository on the assignment page in Canvas. Your repository should be private **but shared with your course grader and/or instructor. Instructions for granting access to your repo for the course grader will be provided by your instructor.**

Note: If you are invoking the Non-Late policy please insert a note at the top of your README file that states “INVOKING THE NON_LATE POLICY” followed immediately by your work log. (The rest of you README should still contain any project specified requirements).